```
# MNIST 手写识别项目四
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import StepLR
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
# 构建模型
class OptimizedNet(nn.Module):
    def __init__(self):
        super(OptimizedNet, self).__init__()
        self.fc1 = nn.Linear(28 * 28, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 10)
        self.relu = nn.ReLU()
    def forward(self, x):
        x = x.view(-1, 28 * 28)
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x
# 数据加载
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)
# 初始化
model = OptimizedNet()
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
# 学习率调度器 - StepLR 按固定步长衰减学习率
scheduler = StepLR(optimizer, step_size=2, gamma=0.5)
# 使用学习率调度器训练
```

```
epochs = 8
for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    for data, target in train_loader:
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    # 每个 epoch 结束后，调度器更新学习率
    current_lr = scheduler.get_last_lr()[0]
    scheduler.step()
    # 第 8 讲：验证
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for data, target in test_loader:
            output = model(data)
            pred = output.argmax(dim=1, keepdim=True)
            correct += pred.eq(target.view_as(pred)).sum().item()
            total += target.size(0)
    print(f'Epoch {epoch + 1}, Loss: {running_loss / len(train_loader):.4f}, '
          f'Test Acc: {100 * correct / total:.2f}%, LR: {current_lr:.5f}')
# 对比：不使用学习率调度器，用固定学习率训练
print("\n 对比实验：固定学习率 0.1 训练")
model_fixed = OptimizedNet()
optimizer_fixed = optim.SGD(model_fixed.parameters(), lr=0.1, momentum=0.9)
for epoch in range(8):
    model_fixed.train()
    for data, target in train_loader:
        optimizer_fixed.zero_grad()
        output = model_fixed(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer_fixed.step()
    model_fixed.eval()
```

```
correct = 0
total = 0
with torch.no_grad():
    for data, target in test_loader:
        output = model_fixed(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()
        total += target.size(0)
print(f'Epoch {epoch + 1}, Fixed LR Test Acc: {100 * correct / total:.2f}%')
```